

CAKECART

Home-bakery orders with pickup slots and custom messages

APP AGENT



DATABASE AGENT



QA AGENT



CAKECART

Search cakes...

Fresh this week



Red Velvet 8"

Serves 10 · AED 190

3 left today



Lotus Cheesecake

Serves 8 · AED 160

3 left today



Chocolate Fudge

Serves 12 · AED 210

3 left today

Pickup slots · Friday 25 Sep

10:00

12:00

CAKECART

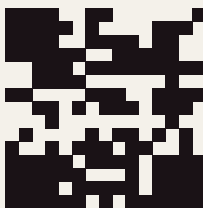
READY FRI

Red Velvet 8"

Pickup: Fri 25 Sep · 14:00

Red Velvet 8" · "Happy Birthday Sara!"

Ref: CC-2H9P-47 · AED 205



CAKECART

Red Velvet 8"

Serves 10 · AED 190

Pickup Fri 14:00

Message on cake

Happy Birthday Sara!

20 / 40 characters

Daily capacity



7 of 10 cakes ordered for Friday

Total

AED 205

Place order

VERCEL DEVELOPMENT PROMPT

Next.js | Neon PostgreSQL | Drizzle ORM | Three-Agent Team

CakeCart Development Specification

Create a team called CakeCart with three agents.

Goal: Build a production-ready home-bakery ordering web app with daily capacity limits, pickup slots, and custom cake messages, and prepare it for deployment on Vercel.

Technology

- Next.js with TypeScript and App Router
- Tailwind CSS
- Neon serverless PostgreSQL from Vercel Marketplace
- Drizzle ORM with migrations
- Secure authentication (email + password, sessions in httpOnly cookies)
- Vercel Functions for backend operations
- Vercel Blob for cake photos and customer reference images
- Payment provider in test mode
- Never expose database or payment secrets in browser code

Agent 1 - App Agent

- Build a polished responsive bakery ordering interface
- Create home, menu, cake details, customise (size, flavour, message), pickup date and slot picker, cart, checkout, order confirmation, my orders, login, and baker dashboard pages
- Let customers browse and filter cakes by category, flavour, dietary tag (eggless, gluten-free, nut-free), size, and price
- Show remaining daily capacity and available pickup slots for each date
- Enforce a minimum lead time (for example 48 hours) and a message length limit (40 characters) on the client and the server
- Show items, customisations, message fee, and final total before payment
- Generate an order confirmation with order reference, pickup time, and QR code for collection
- Allow customers to view and cancel eligible orders (cut-off: 24 hours before pickup)
- Give the baker a dashboard to see orders by pickup day, mark orders as BAKING, READY, and COLLECTED, set daily capacity, and close dates
- Connect every screen to the real database
- Do not use permanent mock data or placeholder buttons
- Optimize the application for desktop and mobile

Agent 2 - Database Engine Agent

Use Neon PostgreSQL and create migrations for:

- users
- products
- categories
- product_categories
- product_options (sizes, flavours)
- dietary_tags
- product_dietary_tags
- daily_capacity
- pickup_slots
- orders
- order_items
- order_customisations
- payments
- audit_logs

Database Rules

- Use UUID primary keys
- Store all timestamps in UTC
- Store money as integer minor units, never floating-point values
- Add foreign keys, validation constraints, indexes, and appropriate cascade rules
- Create a unique constraint on daily_capacity per bakery date
- Store daily_capacity as max_cakes and reserved_cakes; reserved_cakes must never exceed max_cakes (check constraint)
- Store pickup_slots with a per-slot maximum number of orders
- Limit custom messages to 40 characters at the database level
- Store order status as PENDING, CONFIRMED, BAKING, READY, COLLECTED, CANCELLED, EXPIRED, or REFUNDED
- Store payment status separately from order status

Order Transaction

- 1 Begin a database transaction
- 2 Lock the daily_capacity row for the chosen pickup date and the chosen pickup_slot row
- 3 Confirm remaining capacity covers the number of cakes in the cart and the slot still has room
- 4 Increment reserved_cakes and slot usage as a temporary hold with a 10-minute expiration time
- 5 Calculate the price on the server, including size, flavour, and message fees
- 6 Create the pending order
- 7 Commit the transaction
- 8 Confirm the order only after verified payment
- 9 Use payment idempotency keys so retries cannot create duplicate orders
- 10 Reject the order if capacity or slot is no longer available

Create a secure server-side endpoint that releases expired holds and decrements reserved_cakes. Make the operation idempotent so repeated runs are safe. If using Vercel Cron, protect the route with CRON_SECRET.

Add seed data containing sample cakes, sizes, flavours, dietary tags, daily capacity for the next 14 days, and pickup slots.

Agent 3 - QA Agent

- Test registration, login, logout, and authorization
- Test menu browsing, filters, customisation options, and the message character limit
- Test complete orders from menu through confirmation QR
- Attempt to order the last available cake for a date from two sessions
- Confirm only one order succeeds and capacity never goes negative
- Test expired holds and automatic capacity release
- Test minimum lead time and closed dates are rejected on the server, not only in the UI
- Verify size, flavour, and message fees, totals, and payment status
- Test payment success, failure, cancellation, retry, and duplicate webhook delivery
- Test cancel flow, including the 24-hour cut-off rule
- Confirm customers cannot access another customer's orders
- Test baker dashboard permissions and status transitions
- Test database migrations on a clean database
- Test Vercel preview and production builds
- Test desktop and mobile layouts
- Report every bug to the responsible agent and retest each fix

Deployment Requirements

- Deploy the Next.js application to Vercel
- Provision Neon PostgreSQL through Vercel Marketplace
- Use pooled DATABASE_URL for application requests
- Keep migration credentials and production secrets server-side
- Configure separate preview and production environments
- Run migrations safely during deployment
- Configure payment webhook secrets
- Protect scheduled cleanup endpoints
- Add error handling, loading states, empty states, and structured server logs
- Create a README containing local setup, environment variables, migrations, testing, and Vercel deployment instructions

The agents must share a task list and communicate directly. Do not mark the project complete until the database migrations pass, concurrent capacity tests pass, payment flows work in test mode, and the deployed Vercel application works end to end.

How to Build and Go Live - Simple Steps

Follow these in order. Each step is small. If something breaks, paste the error into Antigravity and say: *fix this error*.

Step 1 - Make a folder

- **Windows:** open File Explorer, go to C: then Users then your name, make a new folder and name it **cakecart**
- **Mac:** open Finder, click Go then Home (or press Shift + Command + H), right-click and choose New Folder, name it **cakecart**
- Use small letters and no spaces in the folder name

Step 2 - Open Antigravity

- Open the Antigravity app
- Click **File** then **Open Folder**
- Pick the **cakecart** folder and click Open

Step 3 - Paste the prompt

- Look at the box on the right side of Antigravity
- Copy pages 2 to 4 of this PDF and paste them into that box
- Press Enter and wait. The three agents will build the app
- When it says done, you will see files on the left side

Step 4 - See it on your laptop (my local link)

- In the Antigravity terminal type: **npm install** then **npm run dev**
- Open your browser and go to **http://localhost:3000** (same on Windows and Mac)
- This is your **local link**. Only you can see it. If the app opens, good

Step 5 - Make a GitHub home for the code

- Go to github.com and log in
- Click **New repository**
- Name it **cakecart**, keep it Public, click **Create repository**
- Copy the link that looks like <https://github.com/YOUR-NAME/cakecart.git>

Step 6 - Send the code to GitHub

- Go back to the box in Antigravity and paste this:
- *Connect this project to <https://github.com/YOUR-NAME/cakecart.git> and push all the code to the main branch*
- If it asks you to log in to GitHub, click Allow in the browser
- Refresh your GitHub page. Your files should be there now

Step 7 - Put it on Vercel (my live link)

- Go to vercel.com and click **Continue with GitHub**
- Click **Add New** then **Project**
- Find **cakecart** in the list and click **Import**
- Do not change anything. Click **Deploy**
- Wait 1 to 2 minutes

Step 8 - Add the database (choose Neon, it is the easiest)

- In your Vercel project click the **Storage** tab
- Click **Create Database** and choose **Neon**
- Pick the Free plan and click **Connect to project**
- Vercel now adds the database key for you. You do not type anything
- Go to **Deployments** and click **Redeploy** so the app can see the database

Why Neon?

It is inside Vercel. One click, free, and the connection is done for you. Other databases need you to copy keys by hand.

Step 9 - Fill the database with tables and sample cakes

- In the Antigravity terminal type these one by one:
- **`npx vercel link`** (press Enter and say Yes to the questions)
- **`npx vercel env pull .env.local`**
- **`npx drizzle-kit migrate`**
- **`npm run seed`**

Step 10 - Get your live link

- Go back to Vercel and click **Visit**
- This is your **live link**, for example <https://cakecart.vercel.app>
- Anyone in the world can open it. Send it to your friends

If something goes wrong

- **Vercel build failed** - copy the red text and paste into Antigravity: *fix this Vercel build error*. Then push again (Step 6) and Vercel redeploys by itself
- **App opens but no cakes show** - you skipped Step 9. Do it now
- **Payment does not work** - the test keys are missing. In Vercel go to Settings, then Environment Variables, and add them
- **Local link works but live link does not** - almost always the database. Check Step 8 and Step 9